```
# Monthly Upstream Sync Process

## Purpose

This process syncs bug fixes and improvements from upstream `hass_local_openai_llm` in

Run this **monthly** or when:
- Upstream releases a significant update
- Home Assistant introduces breaking changes
- You discover an upstream bug fix you need

---

## Prerequisites

Confirm your Git remotes are configured:

```bash
git remote -v
```

Expected output:

```
origin    https://github.com/<you>/personality_llm.git (fetch/push)
upstream  https://github.com/skye-harris/hass_local_openai_llm.git (fetch/push)
```

If `upstream` is missing:

```
git remote add upstream https://github.com/skye-harris/hass_local_openai_llm.git
git fetch upstream
```
```

Step 1 – Review Upstream Changes

Purpose: Decide if a sync is worthwhile before starting the merge.

```
# Fetch latest upstream commits (does not modify your code)
git fetch upstream
```

```
# Review what changed since your last sync
git log upstream/master --since="1 month ago" --oneline
```

Look for:

- ✅ Bug fixes (especially in OpenAI API client or tool execution)
- ✅ HA compatibility updates
- ✅ Performance improvements
- ⚠️ Large refactors (may conflict heavily)
- ❌ Features you don't need (can skip)

Decision:

- If no meaningful changes → **stop here**, revisit next month
- If useful changes exist → **proceed to Step 2**

Step 2 – Create Clean Sync Branch

Purpose: Isolate the merge from your stable `main` branch.

```
# Ensure you're on main and it's up to date
git checkout main
git pull origin main

# Create/reset upstream-sync branch
git checkout -B upstream-sync main
```

This creates a fresh `upstream-sync` branch from your current `main`.

Step 3 – Merge Upstream

Purpose: Bring upstream changes into the sync branch.

```
git merge upstream/master
```

If Merge Succeeds (No Conflicts)

Proceed directly to **Step 5 (Testing)**.

If Merge Reports Conflicts

Git will output:

```
Auto-merging custom_components/personality_llm/conversation.py
CONFLICT (content): Merge conflict in conversation.py
```

Proceed to **Step 4 (Resolve Conflicts)**.

Step 4 – Resolve Conflicts

Purpose: Manually decide which changes to keep.

Expected Conflict Files

File	Likelihood	Resolution Strategy
`conversation.py`	🔴 High	Keep your speaker logic; cherry-pick upstream bug fixes
`config_flow.py`	🔴 High	**Always keep yours** (completely rewritten)
`__init__.py`	🟡 Medium	Keep your webhook; merge other upstream changes
`const.py`	🟡 Medium	Merge both (your constants + upstream constants)

File	Likelihood	Resolution Strategy
`manifest.json`	🟢 Low	Accept upstream dependencies; reapply your name/domain

Conflict Resolution Process

For each conflicted file:

```
# Open file in editor
code custom_components/personality_llm/conversation.py
```

Git marks conflicts like this:

```
<<<<<<< HEAD (your fork)
# Your speaker-aware code
speaker = self._get_speaker_from_cache()
user_config = self.config_manager.get_user_config(speaker)
=======
# Upstream code (original, single-user)
# (may include bug fixes or improvements)
>>>>>>> upstream/master
```

Resolution strategy:

1. **Identify the upstream change** (what did they fix or add?)
2. **Keep your speaker logic** (the core modification)
3. **Manually apply upstream improvements** to your version (if applicable)

Example – conversation.py:

If upstream fixed a bug in error handling, apply the fix to your version:

```
# Keep your speaker logic
speaker = self._get_speaker_from_cache()
user_config = self.config_manager.get_user_config(speaker)

# Apply upstream's improved error handling (if present)
```

```
try:
    response = await self._call_openai_api(...)
except TimeoutError:
    # Upstream added better error message - apply it
    _LOGGER.error("OpenAI API timeout after 30s")
    return self._build_error_response(...)
```

Example – config_flow.py:

Always keep yours (no merge):

```
# Accept your version entirely
git checkout --ours custom_components/personality_llm/config_flow.py
```

After resolving each file:

```
git add custom_components/personality_llm/conversation.py
git add custom_components/personality_llm/config_flow.py
```

Complete the merge:

```
git commit -m "chore: merge upstream master ($(date +%Y-%m-%d))"
```

Step 5 – Test Merged Code

Purpose: Ensure upstream changes didn't break your fork.

Minimal Validation Checklist

Run these tests before merging to `main`:

- ☐ **Installation**

  ```
  # Copy to HA custom_components and restart
  cp -r custom_components/personality_llm /config/custom_components/
  # Restart Home Assistant
  ```

- ☐ **Component loads without errors**
  Check HA logs for:

`Setup of domain personality_llm took X seconds`

No errors or warnings.

- ☐ **Baseline conversation works**
  Test: "What time is it?"
  Expected: Response received, no errors in logs
- ☐ **Tool calling works**
  Test: "Turn on the kitchen light"
  Expected: Light turns on, logs show tool execution
- ☐ **Speaker awareness works**
  - POST webhook with speaker metadata
  - Speak as configured speaker
  - Expected: Correct personality prompt used
- ☐ **Multi-speaker config UI works**
  - Add/edit a speaker via config flow
  - Expected: Config saves, no UI errors
- ☐ **No regression in latency**
  Measure response time for a simple query
  Expected: Similar to pre-merge (<2-3s local)

If Tests Fail

Option A – Fix the issue:

- Debug the failure
- Fix in `upstream-sync` branch
- Retest

Option B – Abort the sync:

```
git checkout main
git branch -D upstream-sync
```

Document why the merge failed, revisit later or skip this upstream update.

Step 6 – Merge to Main

Purpose: Apply validated changes to your stable branch.

```
# Switch to main
git checkout main

# Merge the tested sync branch
git merge upstream-sync

# Push to your fork
git push origin main

# Clean up sync branch
git branch -D upstream-sync
```

Your `main` branch now includes upstream improvements while preserving your speaker-aware modifications.

Step 7 – Document the Sync

Purpose: Track what was synced and when.

Create/update `docs/UPSTREAM_SYNC_LOG.md`:

```
## 2024-12-XX – Monthly Sync

**Upstream commits merged:**
- abc1234: Fix timeout handling in OpenAI client
- def5678: Update manifest for HA 2024.12

**Conflicts resolved:**
```

```
- conversation.py: Kept speaker logic, applied timeout fix
- config_flow.py: Kept ours (no merge)

**Testing:**
- ✅ Baseline conversation
- ✅ Tool calling (light.turn_on)
- ✅ Speaker awareness
- ✅ No latency regression

**Issues:** None

**Next sync due:** 2025-01-XX
```

When NOT to Sync

Skip the sync if:

- Upstream has no meaningful changes (run Step 1, find nothing)
- You're mid-feature and `main` is unstable
- Upstream introduced a large refactor you don't need yet

It's better to skip than to merge untested or unnecessary changes.

Quick Reference – Full Sync Commands

```
# 1. Review upstream
git fetch upstream
git log upstream/master --since="1 month ago" --oneline

# 2. Create sync branch
git checkout main
git pull origin main
git checkout -B upstream-sync main

# 3. Merge upstream
git merge upstream/master
```

```
# 4. Resolve conflicts (if any)
# ... manual editing ...
git add <files>
git commit

# 5. Test thoroughly
# ... run validation checklist ...

# 6. Merge to main
git checkout main
git merge upstream-sync
git push origin main
git branch -D upstream-sync

# 7. Document in UPSTREAM_SYNC_LOG.md
```

Notes

- **Frequency:** Monthly is a guideline, not a rule. Sync when it makes sense.
- **Selective merging:** You can choose to ignore upstream changes if they don't benefit your fork.
- **Conflict files:** `conversation.py` and `config_flow.py` will almost always conflict. This is expected.
- **Safety:** The `upstream-sync` branch isolates risk. If the merge breaks things, your `main` is untouched.

This process takes **30-60 minutes per month** and keeps your fork aligned with upstream bug fixes without sacrificing your custom functionality.